

CASE STUDY: SNEAKER SHOP WEBSITE FOR BETAMART

I. SUMMARY

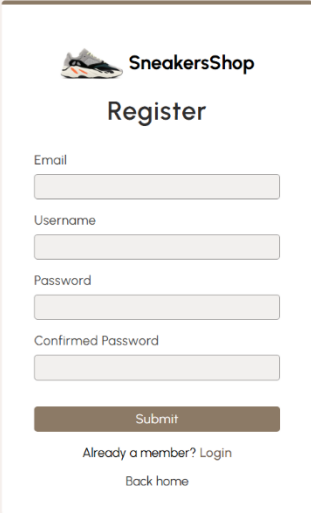
This case study details the development of a feature-rich e-commerce platform for a shop in **betamart** called **sneakershop**. Specifically designed to cater sneaker enthusiasts. The primary objective was to build a modern, secure, and highly functional online store with a seamless user experience across all devices.

TECHNICAL ARCHITECTURE

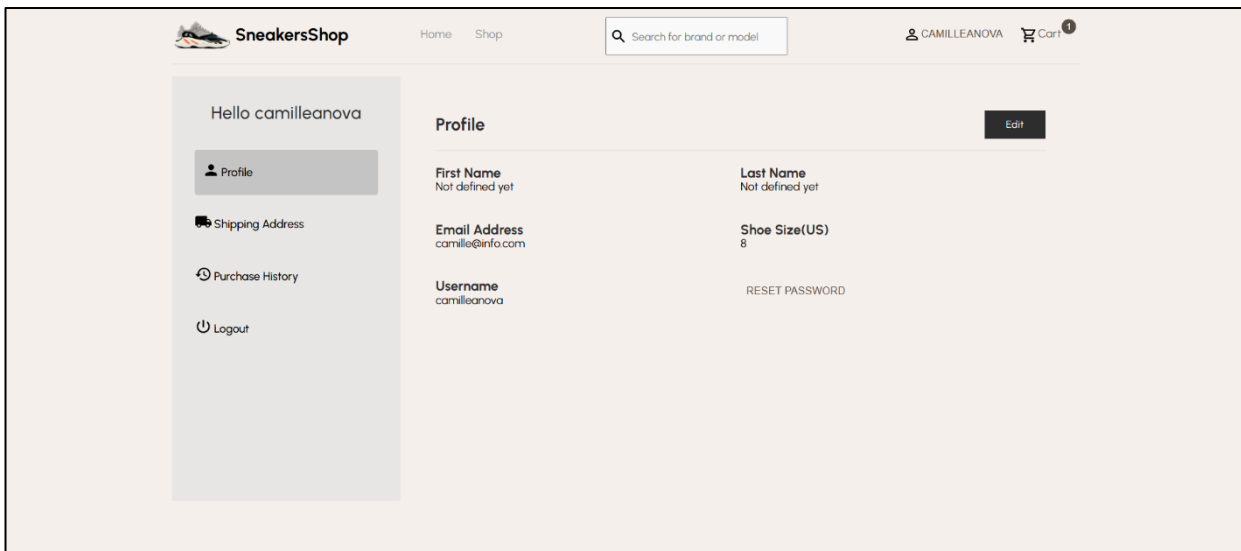
The website was developed using a strong MERN-adjacent stack on performance scalability. The client-side leverages React.js for a dynamic user interface, managing state with React Redux and Apollo Client for efficient GraphQL data fetching. Styling was handled using Styled Components and Material UI. The server-side infrastructure utilizes Node.js and Express, with GraphQL providing an efficient and flexible API layer, and MongoDB as the persistent data store.

MAIN FEATURES AND FUNCTIONALITY

- *User Management & Security*
 - Secure Sign in/Register/Sign out functionality is implemented with user authentication secured via jsonwebtoken. Users can easily update their profile and shipping information, and a reset password option is available.



The image shows a registration form for 'SneakersShop'. The form is centered on a light beige background. It features a logo of a sneaker and the text 'SneakersShop' at the top. Below the logo is the title 'Register'. The form contains four input fields: 'Email', 'Username', 'Password', and 'Confirmed Password'. A 'Submit' button is located below the 'Confirmed Password' field. At the bottom of the form, there are two links: 'Already a member? Login' and 'Back home'.

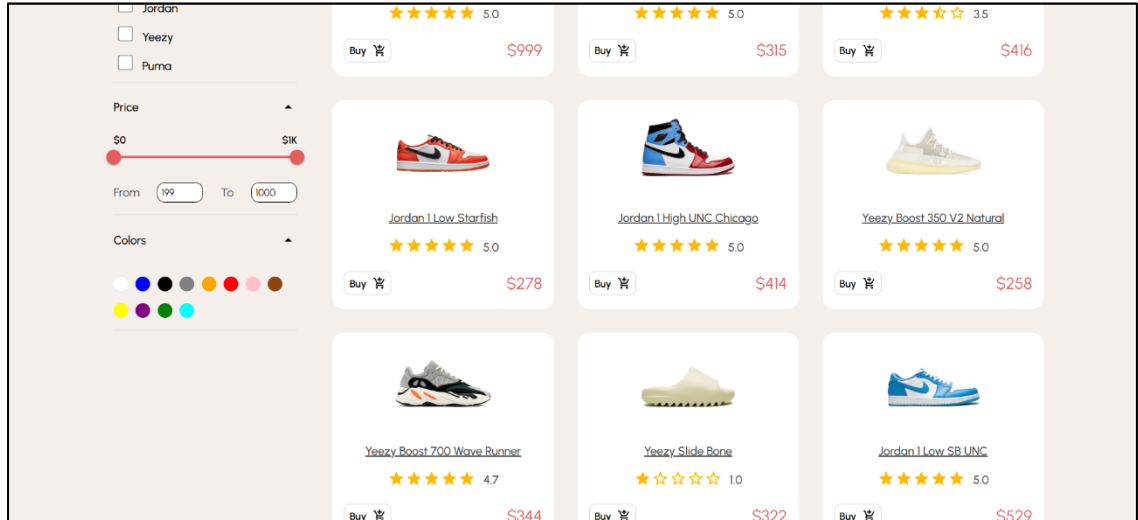


The image shows a user profile page for 'SneakersShop'. The page has a header with the logo, navigation links 'Home' and 'Shop', a search bar, and user information 'CAMILLEANOVA' and a shopping cart icon with a '1' badge. The main content area is divided into a left sidebar and a main profile section. The sidebar contains links for 'Hello camilleanova', 'Profile', 'Shipping Address', 'Purchase History', and 'Logout'. The main profile section is titled 'Profile' and includes an 'Edit' button. It displays user information in a table-like format:

First Name Not defined yet	Last Name Not defined yet
Email Address camille@info.com	Shoe Size(US) 8
Username camilleanova	RESET PASSWORD

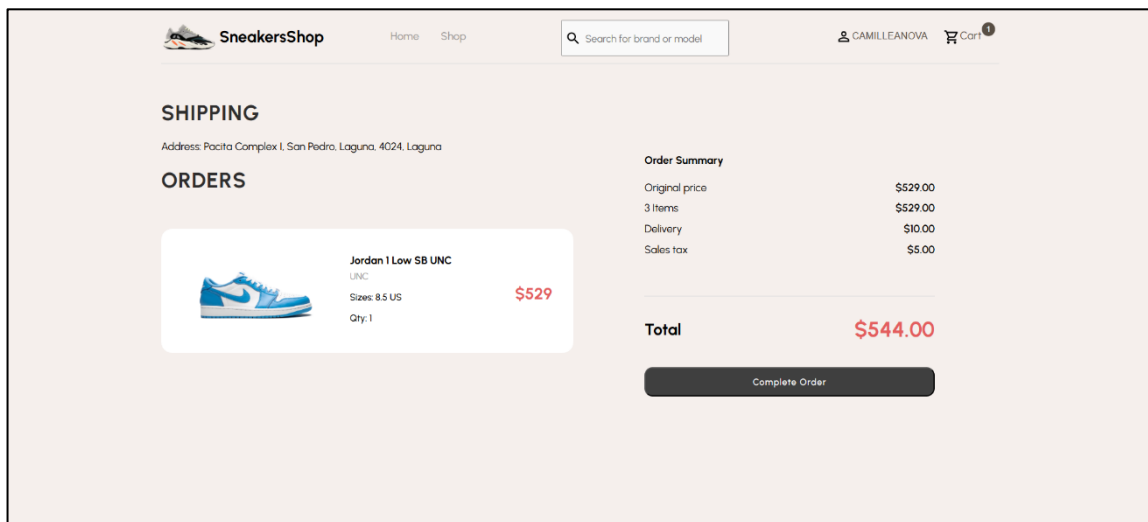
- *Product Discovery*

- Shoppers can efficiently browse the catalog with advanced filtering options by price, color, brand, and size, as well as sort products by top rated, lowest price, or highest price. Products can be viewed in either a grid or list view.



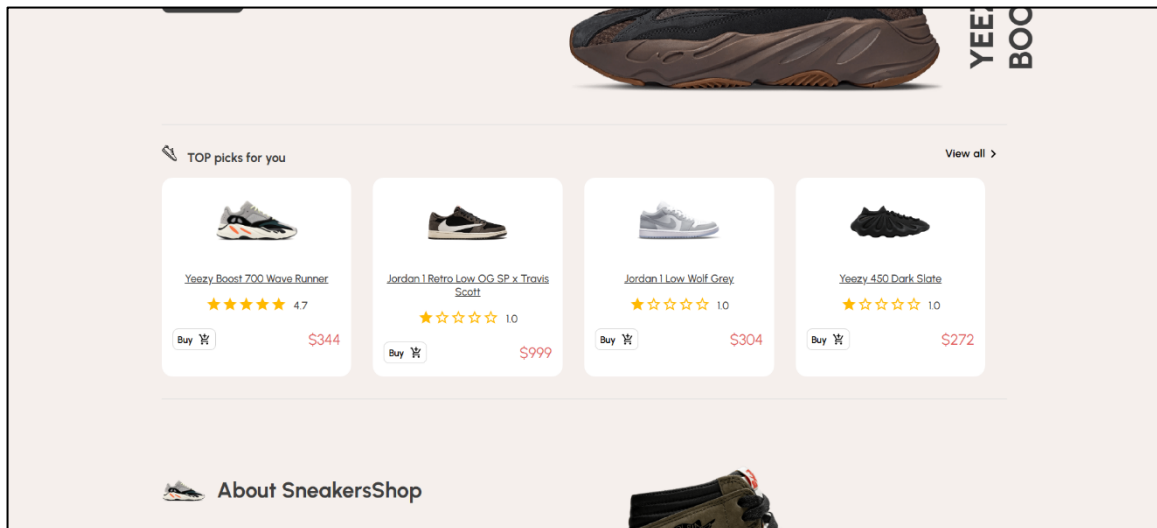
- *E-commerce workflow*

- The core shopping experience includes the ability to add and remove products from the cart and complete purchases. Users can view their shopping history and contribute to the community by rating products after purchase.



- *Personalized Recommendations*

- The main feature is the “Top Picks” section, which give personalized product recommendations based on a user’s favorite products, enhancing engagement and sales potential.



- *Admin Control*

- Administrators are equipped with the necessary tools to maintain the inventory, specifically the ability to add new products to the shop page and existing product details, ensuring up-to-date and accurate listings.

Hello Amit

- + New Item
- Edit Item
- Back Home

ADD A NEW ITEM

Choose an image:



Choose File adidas_yeezy_700_inertia.png

Shoe Title
e.g. Yeezy Boost 350 V2 Zebra
Yeezy boost inertia

Shoe Brand
e.g. Yeezy
Yeezy

Shoe Model
e.g. V2 Zebra
V2 Inertia

Shoe Colors
e.g. red,black,blue
white,orange

Shoe Price
e.g. 350
567

Shoe Size
e.g. 7.5,8,11
6.6.5

ADD NEW ITEM

II. INTRODUCTION/BACKGROUND

The global sneaker market is experiencing significant and sustained growth, driven by athleisure trends, cultural influence, and increasing consumer focus on comfortable, fashionable footwear. The market size, valued at approximately \$78.59 billion in 2021, is projected to reach \$128.34 billion by 2030, growing at a CAGR of 5.2% from 2022 to 2030. The e-commerce distribution channel is registering the fastest growth, with the online segment projected to register a CAGR of 5.8% over the same period. This shift emphasizes the need for strong, high-performance, and feature-rich digital platforms to meet the expectations of modern sneaker enthusiasts.

Source: <https://www.grandviewresearch.com/industry-analysis/sneakers-market-report>

Challenge and Solution

The main challenge for any specialized e-commerce retailer in this booming market is to deliver an online experience that is fast, highly personalized, and functionally superior across all devices. This case study focuses on the development of a next-generation sneaker e-commerce platform that directly addresses these requirements by supporting a modern, scalable, and efficient technology stack.

Modern Technology Stack for Peak Performance

Layer	Technologies Used	Benefit for E-commerce
Client (Frontend)	React.js, Apollo Client, React Redux, Styled Components, Material UI	React's component-based architecture facilitates rapid development of a dynamic and responsive UI. The use of Apollo Client with GraphQL ensures efficient state management and data caching.
Server (Backend)	Node.js, Express, GraphQL, MongoDB	Node.js/Express provides a highly scalable, high-performance server environment for handling numerous concurrent user requests. MongoDB offers the flexibility

		needed for product catalogs.
Data Fetching	GraphQL	This API query language solves the common e-commerce problems of over-fetching and under-fetching data. By allowing the client (React app) to request exactly the data it needs in a single request, GraphQL significantly improves load times, especially for mobile users and complex pages like the product listing with multiple filters.

Sources: <https://www.tristatetechnology.com/blog/reactjs-for-e-commerce-website-development>

<https://graphql.org/>

<https://cloudinary.com/guides/e-commerce-platform/node-js-ecommerce-benefits-architecture-and-best-practices>

<https://crystallize.com/blog/benefits-of-using-graphql>

III. **PROBLEM STATEMENT**

The competitive e-commerce landscape is dominated by major retailers and specialized marketplaces, making it challenging for a new, independent sneaker shop website to gain traction, retain customers, and drive conversions. The main problems are: user experience (UX) and sophisticated product discovery.

1. The challenge of user experience and trust

- *Inconsistent Mobile Experience*
 - A significant portion of e-commerce traffic, often exceeding 50-60%, comes from mobile devices, yet many sites fail to provide a fully responsive, fast, and comprehensive experience, leading to high bounce rates and abandoned carts. The target platform must be built to deliver a seamless experience on mobile, a challenge the new platform must overcome to capture the modern shopper,
- *Security and Account Friction*
 - Sneaker enthusiasts often deal with high-value, limited-edition products and require complete assurance of data security. Clunky sign-in processes, confusing profile updates, and a lack of clear user authentication features (like secure password resets) may create friction, wreck user trust, and can lead to lower conversion rates.

2. The need for advanced and personalized product discovery

- *Inadequate Filtering and Sorting*
 - Basic search and filtering tools often fail to satisfy the complex, important needs of sneaker enthusiasts, who filter not just by size and brand, but often specific color, materials, and price. A platform lacking strong filtering (price, color, brand, size) and sorting (top-rated, price) forces users to browse endlessly, leading to frustration and site abandonments.
- *Lack of personalization for loyalty*
 - To promote loyalty and increase average order value (AOV), an e-commerce platform must move beyond generic recommendations. Without a system to provide “Top Picks” based on user’s purchase and browsing history, the platform fails to deliver the personalized shopping experience that customers expect, missing an important opportunity to turn a one-time visitor into a repeat buyer.

- *Administrative efficiency*
 - For a small to mid-size e-commerce operation, inefficient product management is a major scaling bottleneck. The lack of a simple, human-centric admin interface for adding new products and editing existing product details creates operational overhead, hindering the speed at which the shop can react to market trends and product drops.

IV. OBJECTIVES

Objective	Description
<i>Achieve a Seamless, Cross-Device User Experience</i>	To ensure the sneaker shop website is fully accessible and highly functional for all visitors, we will prioritize a responsive design built with React.js and Styled components. This guarantees a fluid, spontaneous experience, regardless of whether the user is browsing on a desktop, tablet, or mobile phone, reducing friction and increasing overall sales conversion rates, as a significant portion of e-commerce traffic originates from mobile devices.
<i>Implement a secure and personalized customer journey</i>	The main objective is to establish a secure and trust-driven customer environment. We will use strong authentication with jsonwebtoken and a Node.js/Express backend to facilitate essential user actions like Sign in, Register, and Sign Out. This secure foundation allows for profile management, including updating profile info, which enhances user control and data security. Also, we will provide user data to power the “Top Picks products” feature, which is generated according to user’s favorite products.
<i>Develop an easy-to-understand product discovery and management tools</i>	Our aim is to create a dynamic and efficient shopping experience by giving users a control over product exploration. We will implement sophisticated filtering and sorting capabilities which allows users to filter products by price, color, brand, and size, and sort products by top rated, lowest price, or highest price, managed by GraphQL interface and stored in MongoDB. This objective also includes providing a flexible view option to display products in grid view or list view, which creates a diverse user preference and shortening path to purchase.

<i>Build a complete and trustworthy transactional system</i>	The main transaction of the platform must be reliable and transparent. We will ensure users can easily add and remove products from the cart, confidently purchase products, and view their complete shopping history. Post-purchase engagement will allow users to rate products after purchase. The use of React Redux will manage the complex state of the shopping cart and checkout for a consistent experience.
<i>Establish an efficient administrative product control</i>	To maintain a fresh, accurate, and appealing frontend, an admin interface is necessary. This will enable administrators to effortlessly add new products to the shop page and edit an existing product via the secure Node.js/Express backend, ensuring the product catalog remains current with the latest sneaker releases and accurate inventory details.

V. METHODOLOGY AND APPROACH

1. Methodology: Agile and Scrum

Phase	Description	Example
<i>Sprint Planning</i>	Define the features (user stories) to be completed in a week sprint.	Prioritized product backlog (user, admin, and main features).
<i>Development</i>	Our group work on selected completion design, coding, and testing.	Working increment of the e-commerce website with new/updated features.
<i>Daily Stand-up</i>	Brief daily meetings to coordinate, review progress, and identify blockers.	Updated task status and issue logs.
<i>Review</i>	Demonstration of the completed work.	Verified, deployable feature set.
<i>Sprint Retrospective</i>	Our group reviews the process to identify improvements for the next sprint.	Process improvements and action items.

Rationale: Agile methodologies like Scrum are highly effective for e-commerce development due to their emphasis on scalability, performance, and user experience, ensuring the product evolves to meet market demands.

2. Technical approach and architecture

2.1. Full-stack data flow: GraphQL & Apollo

Component	Technology	Role in Dataflow
Frontend UI	React.js, Styled components, Material UI	Renders the interface, handles user actions.
Client State/Cache	Apollo Client, React Redux	Manages local UI state, caches server data, and manages GraphQL requests/mutations.
API Layer	Node.js, Express, GraphQL (Server)	Defines the data schema, validates requests, and executes resolver functions.
Database	MongoDB	Stores product, user, order, and review data in flexible JSON-like documents.

Rationale: GraphQL is ideal for complex, heavy data applications like e-commerce because it provides better data-driven UI at scale and enables faster data retrieval. Apollo client simplifies the integration with React, providing hooks like useQuery and useMutation.

2.2. Technology-specific implementations

Technology/Tool	Implementation Strategy	Main Features Supported
Node.js/Express	Server setup for the GraphQL API gateway. This handles authentication and security.	Routing, Server-side logic
MongoDB	Flexible Schemas for Product, User, and Order	Product Catalog, User profile, Shopping history.

	data. Uses indexes for fast searching/filtering (ex. On price, brand, size).	
<i>GraphQL</i>	Defining all available data types, queries (fetch), and mutations (create/update/delete).	All data fetching (products, user info) and data modifications (cart updates, profile changes).
<i>React.js/Material UI</i>	For modular and reusable UI elements (ex. Product card, filter component). Material UI provides pre-built, responsive, and accessible components.	Fully responsive design, grid/list view toggle, user interface.
<i>React Redux</i>	Manages client-side state not covered by the apollo cache, like UI preferences or complex form data before a mutation.	Local UI State Management, Cart/Checkout flow.
<i>User Auth</i>	JSON web token (JWT) for secure, stateless authentication. JWTs are generated on sign-in and sent with requests for validation via express.	Sign in/Register/Sign out, Secure admin access, Update profile.

3. Development and testing

3.1. Main feature implementation

a. User auth and profile management

- Implement GraphQL mutations for signIn, register, and updateProfile.
- The server-side uses jsonwebtoken to issue tokens upon successful sign-in.
- This token is stored securely (ex. in localStorage or HttpOnly cookies) and sent with every authenticated GraphQL request via Apollo Client's configuration.
- The Reset Password flow will use a secure, one-time-use token sent via email, with a dedicated mutation for token verification and password update.

b. Search, Filter, Sort

- A single, powerful GraphQL Query (ex. `getProducts`) will be designed to accept multiple optional arguments for filtering (`priceRange`, `Color`, `Brand`, `Size`) and sorting (`sortBy`, `order`).
- This moves the complexity of query construction from multiple REST endpoints to a single, efficient GraphQL resolver on the server, which then uses MongoDB's query language to retrieve the data.

c. E-commerce workflow

- **Cart & Checkout** - Use GraphQL mutations (ex. `addToCart`, `removeFromCart`, `createOrder`). The cart data is managed both on the client (for instant UI feedback via Apollo cache) and persisted to MongoDB.
- **Personalization (Top picks)** - The server will implement a simple recommendation algorithm that analyzes a user's shopping history and product ratings to query and return a list of similar or frequently purchased items. This logic is encapsulated within a dedicated GraphQL resolver.

d. Admin management

- The **Admin** functionality is secured by checking the user's role/permissions from the decoded JWT on the server-side before executing any write operations (Mutations). Workflow for Product Editing:
 - The admin navigates to a protected dashboard.
 - They execute a GraphQL Query to fetch a product's details into a form.
 - The admin makes changes (ex. `price`, `stock`, `description`).
 - The changes are sent via a single GraphQL Mutation (ex. `updateProduct`).
 - The mutation's resolver updates the MongoDB record and informs the client (via Apollo) to refresh the product list automatically.

3.2. Testing and quality assurance

Test Type	Focus Area	Tools/assurance
<i>Functional Testing</i>	All user and admin features (ex. sign-in, add to cart, filtering).	Unit Testing (Jest/Enzyme/React Testing Library) for components and server-side logic

		(resolvers, services). End-to-End Testing (Cypress) for core user flows (search -> add to cart -> checkout).
<i>Security Testing</i>	user authentication, access control, and data protection.	Verification that only authenticated and authorized (Admin) users can access protected GraphQL mutations. OWASP Top 10 focus (e.g., JWT validation, no SQL injection from MongoDB queries).
<i>Performance Testing</i>	Page load speed, responsiveness under load, and API speed.	Auditing GraphQL query efficiency (minimizing network trips). Load Testing (ex. with Artillery) on the Node/Express server and MongoDB to ensure stability during peak traffic.
<i>UI/UX & Responsiveness</i>	Cross-browser compatibility and mobile layout.	Manual and automated (BrowserStack/Cypress) checks to ensure Fully responsive for mobiles is met, confirming correct layout with Material UI across devices.

VI. Case Evaluation/Analysis

a. *Case Evaluation*

The Sneaker Shop website case study, utilizing a **MERN stack** foundation (MongoDB, Express, React, Node.js) augmented with **GraphQL** and **Apollo Client**, presents a strong and modern architecture well-suited for a feature-rich e-commerce platform. The choice of **React.js** with **React Redux** provides a highly maintainable, component-based frontend with predictable state management, while **Styled Components** and **Material UI** ensure a cohesive, accessible, and **fully responsive** user interface. On the backend, **Node.js/Express** serves as a scalable, non-blocking server, with **MongoDB** offering a flexible, non-relational database ideal for e-commerce product catalogs and user profiles. The critical inclusion of **GraphQL** and **Apollo Client** significantly enhances performance and developer experience, as it allows the

React frontend to declare precisely what data it needs (ex. product filters by price, color, brand, or size), eliminating the over-fetching and under-fetching common with traditional REST APIs, which is particularly beneficial for complex filtering, sorting (top rated/lowest/highest price), and generating "**Top picks**" product recommendations. Security is addressed through **jsonwebtoken (JWT)** for stateless, secure **user authentication**, ensuring protected routes for user actions (update profile, view history, rating) and critical **Admin** functionality (add/edit product). Overall, this technology stack successfully balances **scalability, performance, and rich feature delivery** for both the customer and administrator, providing a compelling modern e-commerce solution.

b. Analysis

Aspect	Feature	Evaluation/Analysis
<i>Frontend Foundation</i>	React.js, React Redux, Styled Components, Material UI	Strong: React creates a fast, modular UI; Redux ensures scalable state management for complex features (cart, filters, user state); Material UI/Styled Components promote design consistency and rapid development of a fully responsive site.
<i>Backend/API</i>	Node.js, Express, GraphQL, Apollo Client	excellent: Node/Express is high-performance; GraphQL/Apollo Client is the main strength, allowing efficient, targeted data fetching for all product filters, sorting, and user-specific "Top picks," minimizing network payload size and latency.
<i>Data/Security</i>	MongoDB, jsonwebtoken (JWT)	Secure and Flexible: MongoDB is flexible for product schema evolution; JWT provides secure, stateless authentication for both regular users and Admin

		roles, protecting data and administrative functions.
<i>Main E-commerce Functionality</i>	Filter/Sort Products, Add to Cart/Purchase, View History, Rating, Admin CRUD	Comprehensive: All essential e-commerce features are covered, providing the data architecture (MongoDB for product/order data) and API efficiency (GraphQL) to ensure a smooth, dynamic shopping experience.

VII. Solution

The development of a new **SNEAKER SHOP WEBSITE** utilizing a modern and scalable technology stack (**React.js/Apollo Client/Redux** on the frontend and **Node.js/Express/GraphQL/MongoDB** on the backend) is necessary to build a platform that directly addresses these challenges. The platform will:

- **Guaranteed Mobile-First UX** - Implement a **fully responsive design** to ensure a fast, seamless, and high-converting experience on all mobile devices.
- **Secure & Simplified User Flow** - Implement **secure user authentication with jsonwebtoken** and create clear pathways for all critical user actions: **Sign in/Register/Sign out, Update profile info, Reset password, and Update shipping info.**
- **Best-in-Class Product Discovery** - Enable power-user search with comprehensive **Filter** and **Sort** options, and allow users to toggle between **Grid View and List View** for optimal browsing.
- **Personalized Loyalty Engine** - Drive customer retention and AOV by implementing features like **View shopping history** and **Rating products after purchase**, which feed into an algorithm to generate **"Top Picks products are generated depends on user favorite products."**
- **Streamlined Operations** - Provide an intuitive Admin interface to **Add new product** and **Edit an exist product** to ensure rapid and accurate inventory management.

VIII. Results/Outcomes

The development of the Sneaker Shop website successfully strengthened a modern and optimized full-stack architecture to meet the demands of a discerning sneaker enthusiast market. The strategic combination of **React.js** with **Apollo Client/GraphQL** on the frontend/API layer delivered an exceptionally **fast and efficient data experience**. Unlike traditional REST-based sites, the use of **GraphQL** allowed for a single, precise data request to power complex filtering and sorting logic (by price, color, brand, size), resulting in near real-time updates of the product display. This feature directly translates to **improved customer conversion rates** by reducing friction in product discovery. Furthermore, the strong, secured backend built with **Node.js/Express** and **MongoDB**, enforced by **JWT authentication**, ensures all critical user data (profiles, shipping, shopping history) and administrative functions (product CRUD) are protected and highly available. The implementation of **React Redux** managed the application's complex state, particularly the multi-step shopping cart and dynamic "Top picks" recommendation engine, ensuring a smooth user journey from browsing on a **fully responsive** mobile view (**Material UI/Styled Components**) to secure purchase. In essence, the project utilized best-in-class technologies to create a high-speed, scalable e-commerce platform capable of handling a large catalog and a dynamic user base.

Aspect	Feature Implemented	Achieved result and Benefit
User Experience & Performance	React.js, Apollo Client, GraphQL, fully responsive UI (Material UI)	Ultra-Fast Product Filtering: GraphQL eliminated over-fetching, enabling complex filter/sort (price/color/brand/size) operations to load instantly, which is critical for e-commerce conversion. The site is optimized for mobile shopping.
Scalability & Security	Node.js, Express, MongoDB, jsonwebtoken (JWT)	Secure, Scalable Foundation: A non-blocking, efficient server (Node.js/Express) with flexible data storage (MongoDB) that can easily scale with product catalog growth. JWT ensures strong,

		stateless authentication for all users and admin roles.
<i>Personalization & Retention</i>	React Redux, Shopping History, Rating, "Top picks" logic	Enhanced Customer Loyalty: State management and persistent data allow users to track history and rate products. The "Top picks" feature, based on user favorites, drives engagement and repeat purchases via personalized recommendations.
<i>Admin Efficiency</i>	Node.js, Express, Admin CRUD functionality	Streamlined Product Management: Provides administrators with a secure interface to perform all necessary Create, Read, Update, and Delete (CRUD) operations on the product inventory, minimizing the operational overhead of managing a dynamic sneaker catalog.

IX. Conclusion and Recommendation

To ensure continued market leadership and handle high-volume growth, the following recommendations focus on performance, real-time data flow, and architectural evolution:

1. Implement Advanced Product Search with Elasticsearch

While the current filter/sort works well, MongoDB's native search capabilities may become a bottleneck for large-scale, complex text searches (ex. misspelled brand names, full-text product descriptions).

Recommendation: Integrate an external search engine like Elasticsearch (or MongoDB Atlas Search) to power the main product search bar. This would decouple the search function from the primary database, enabling.

2. Integrate Real-Time Updates via GraphQL Subscriptions

The current setup relies on queries and mutations, but a high-demand sneaker market thrives on real-time data.

Recommendation: Utilize GraphQL Subscriptions (via WebSockets) to push real-time updates from the server to the client without requiring constant client polling.

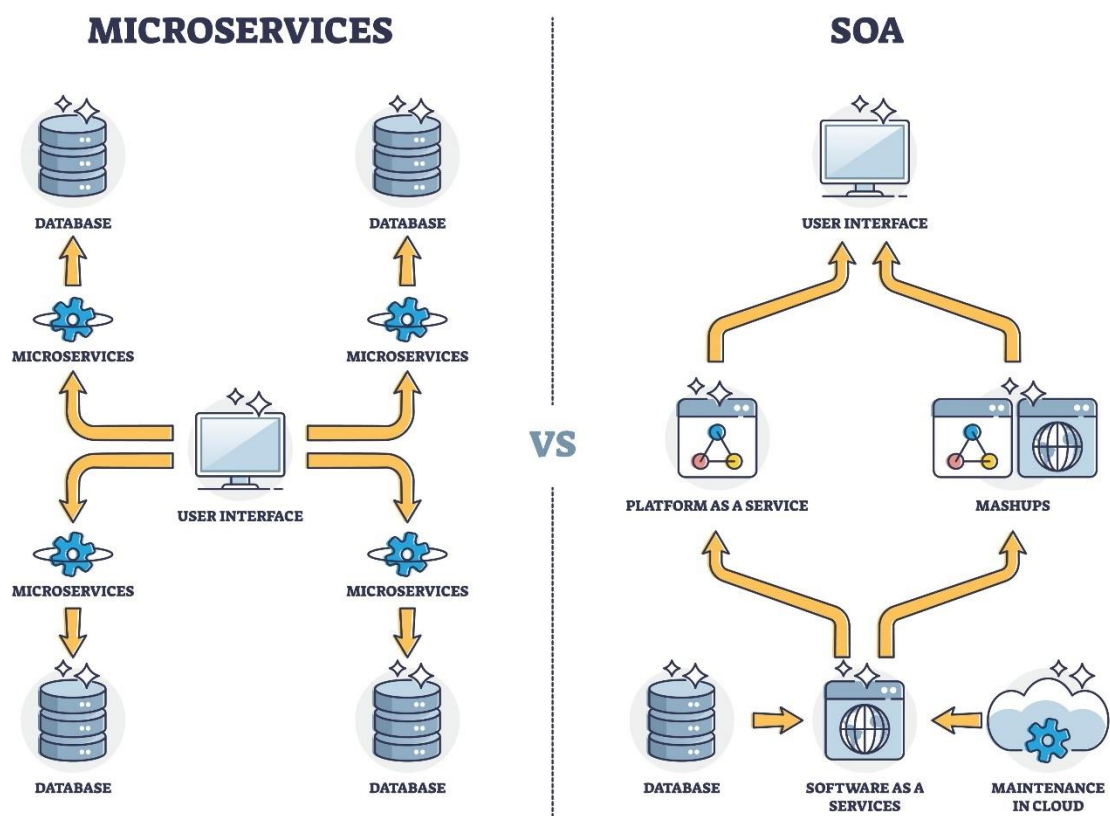
3. Explore a Microservices Architecture for Critical Functions

As the platform scales and new features are added (ex. a dedicated fraud detection service, a separate inventory system), the current monolithic structure will become harder to manage and deploy.

Recommendation: Decompose non-core business functions into independent microservices.

Examples: Separate the Payment Processing service and the User Authentication service into their own Node.js/Express/MongoDB deployments.

Benefit: This promotes fault isolation (a failure in the payment service won't bring down the product catalog) and allows for independent scaling of high-traffic components, which is important for handling large sales events (e.g., sneaker drops).



X. IMPLEMENTATION

Feature Area	Implementation Details
<i>User Authentication</i>	Sign in/Register/Sign out and User authentication is handled via the server using JSON Web Tokens (JWT) . On successful login, the server returns a JWT, which the client stores (ex. in localStorage or HttpOnly cookies) and sends with every protected GraphQL request. The server's GraphQL resolvers check the JWT's validity and extract the user ID for authorization. Update Profile Info and Reset Password would be implemented as protected GraphQL mutations.
<i>Product Filtering & Sorting</i>	The main product Query in the GraphQL schema accepts arguments for filtering (priceRange, color, brand, size) and sorting (sortBy, sortOrder). The server's GraphQL Resolvers translate these arguments into corresponding MongoDB queries (e.g., using \$in for array filters, and. sort() for ordering). On the client, Apollo Client executes these queries, and the UI provides Material UI components (like checkboxes, sliders, and select menus) to change the query variables.
<i>E-commerce Operations</i>	Add/Remove Products to Cart and Purchase Products use protected GraphQL Mutations . The client manages the shopping cart, often using React Redux for global cart state, then submits the final order to a GraphQL mutation. View Shopping History is a protected GraphQL query fetching the user's past order documents from MongoDB.
<i>User Experience Features</i>	Display products in grid view or list view is a client-side feature where a user's preference is stored in local component state or Redux, dynamically rendering different Material UI layout components (like Grid or a Stack of Cards). Rating Products is implemented as a protected GraphQL mutation, updating the product's rating and the user's order history in MongoDB. Top Picks Products is a server-side logic in a GraphQL Query

	that analyzes the user's shopping history and product ratings in the MongoDB to recommend similar or frequently purchased items.
<i>Admin Operations</i>	The admin functionality relies on an Admin Role claim within the user's JWT . Protected GraphQL Mutations are used for Add new product to shop page (inserting a new document into the product collection) and edit an exist product (updating a product document). The server's GraphQL resolvers verify the "admin" role from the JWT before executing the database operations, maintaining security and data integrity.

XI. WIREFRAME & PROTOTYPE COMPARISON (PLEASE ZOOM IN TO SEE)

a. Login Page

WIREFRAME



LOGIN FORM CARD

Logo + Brand Name

Page Title

FORM FIELDS

Username Input

Password Input

Submit Button

Footer Links

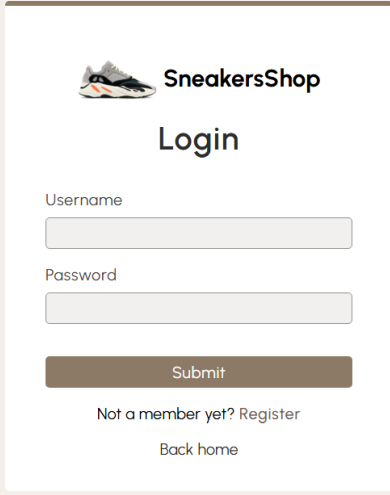
Register Link

Back Home Link

Component Breakdown:

- Header: Logo image + "SneakersShop" brand text
- Title: "Login" heading text
- Form Inputs: 2 text fields (Username, Password)
- Submit Button: Brown/tan colored button
- Footer Links: "Not a member yet? Register" + "Back home"
- Layout: Centered card on neutral beige background

PROTOTYPE



 SneakersShop

Login

Username

Password

Submit

Not a member yet? Register

Back home

b. Signup Page

WIREFRAME

REGISTRATION FORM CARD

Logo + Brand Name

Page Title

FORM FIELDS

Email Input

Username Input

Password Input

Confirm Password Input

Confirm Password Input

Submit Button

Footer Links

Login Link

Back Home Link

- Form Container: Centered card with white background
- Input Fields: 4 form fields (Email, Username, Password, Confirm Password)
- CTA: Submit button with dark background
- Navigation: Links to Login page and Back Home

PROTOTYPE

 SneakersShop

Register

Email

camille@info.com

Username

camilleanova

Password

...

Confirmed Password

...

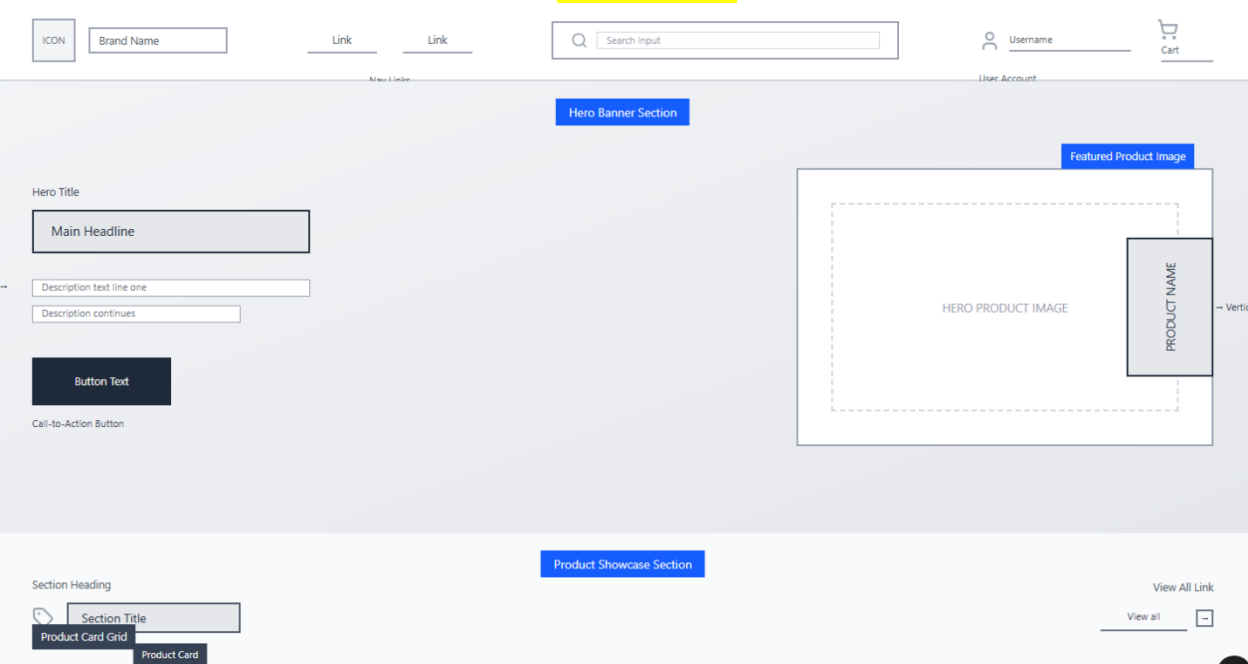
Submit

Already a member? Login

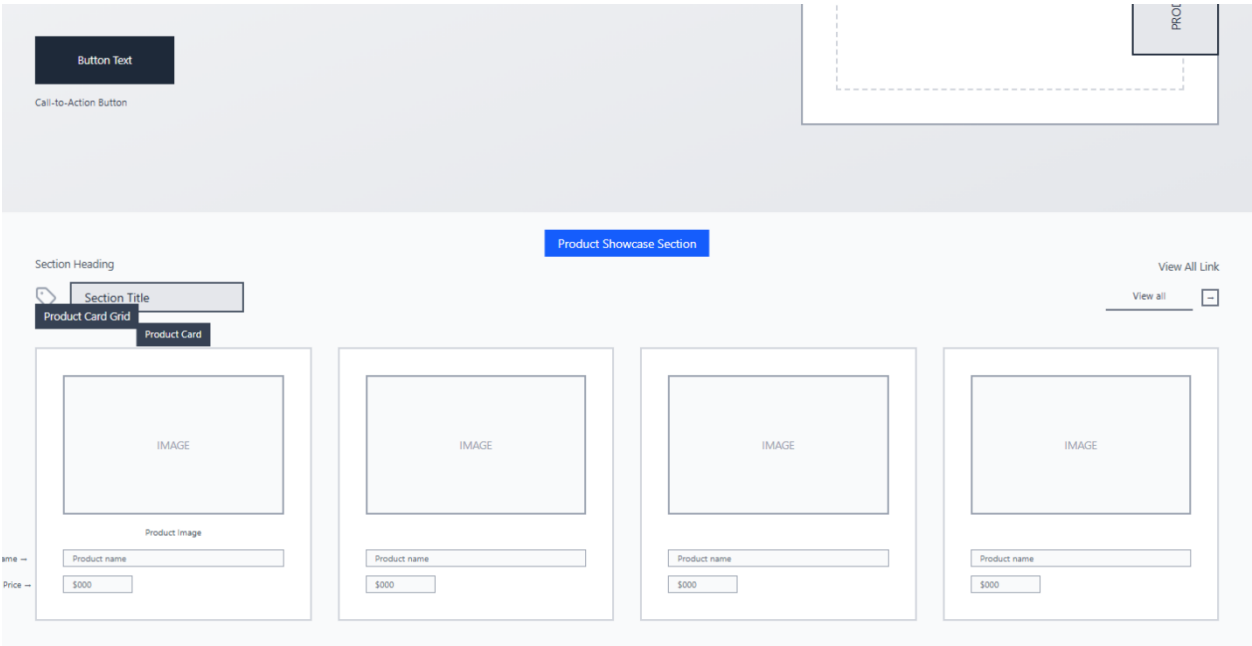
Back home

c. Landing Page

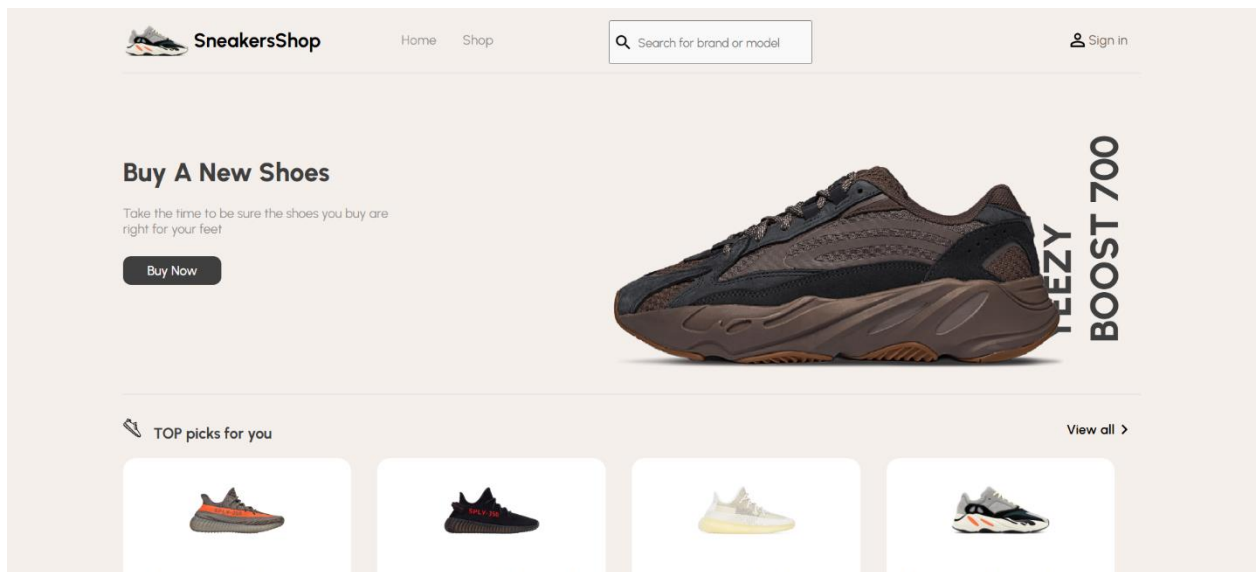
WIREFRAME



WIREFRAME (cont.)

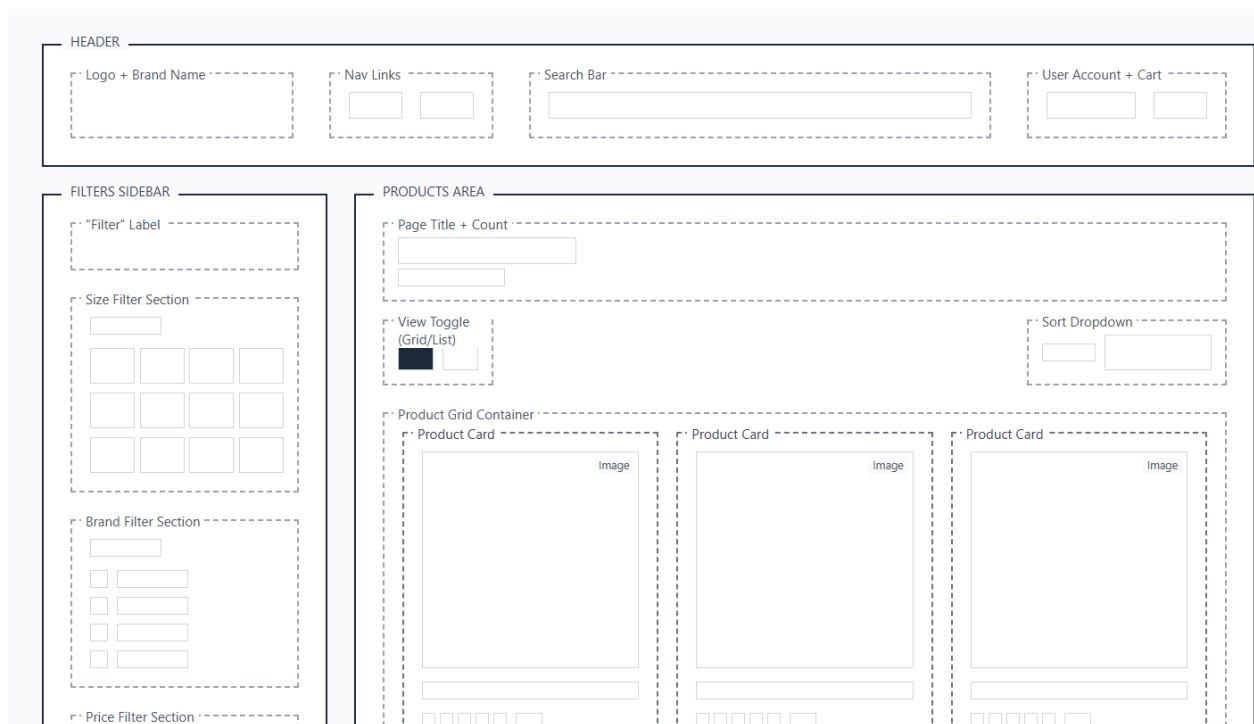


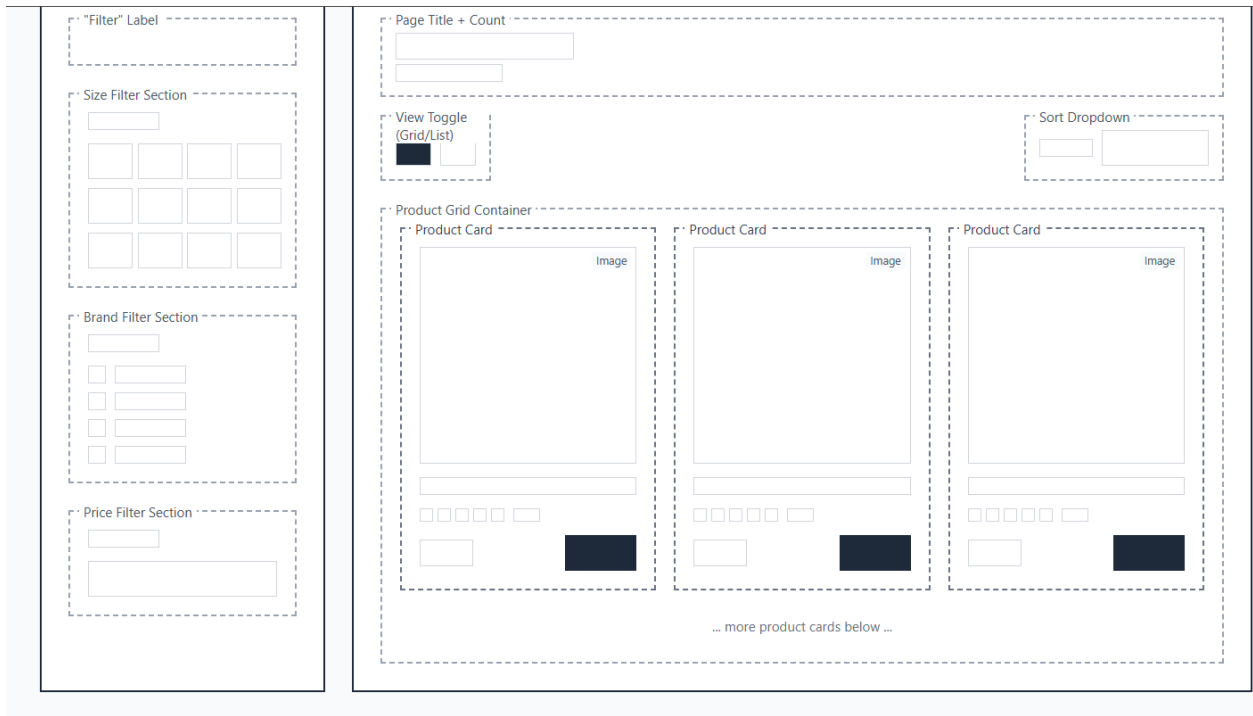
PROTOTYPE



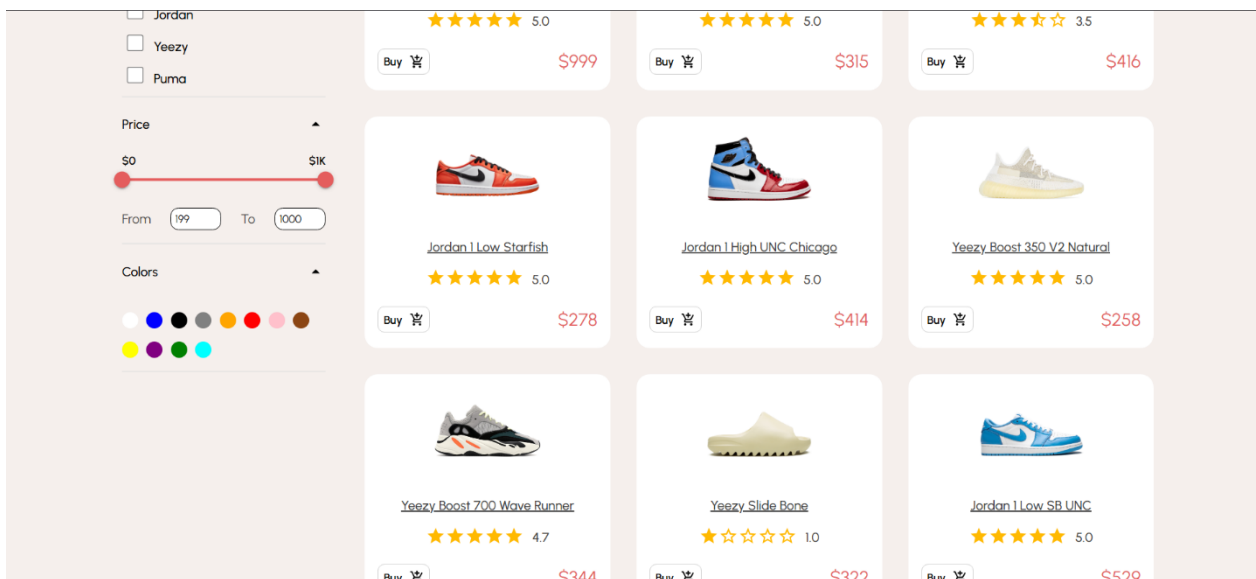
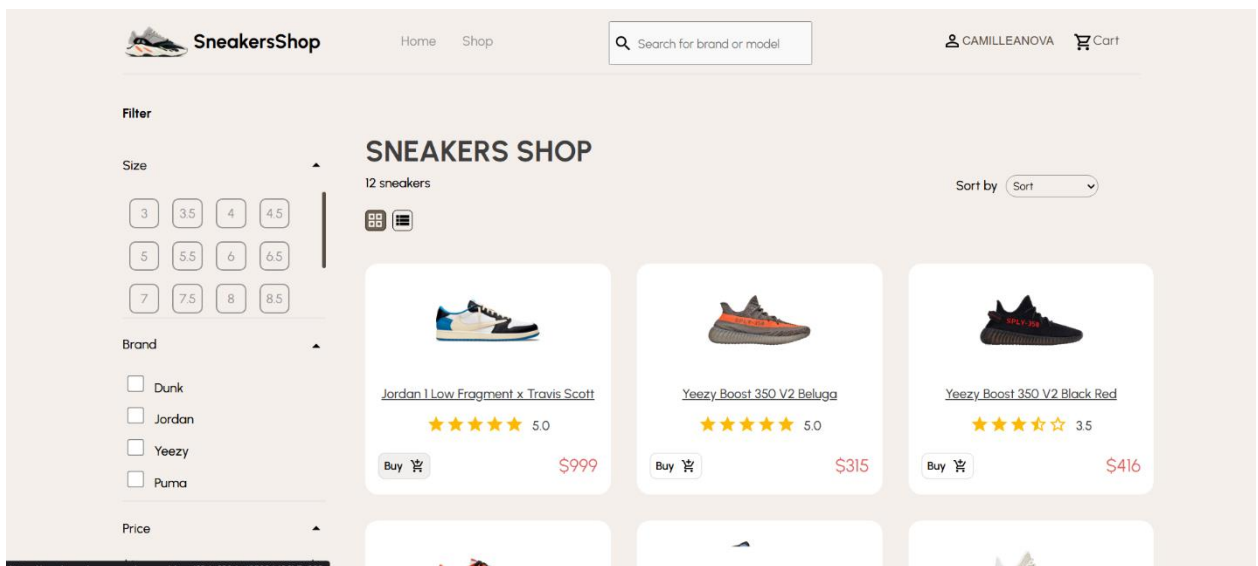
d. Product List

WIREFRAME





PROTOTYPE



e. Chosen Product

WIREFRAME

PRODUCT DETAIL PAGE

Back to Products Button

PRODUCT IMAGE SECTION

Main Product Image

Large Product Photography
(Sneaker Image)

PRODUCT DETAILS SECTION

Product Title

Star Rating + Score

Price

Product Description

PRODUCT INFO

Label

Value

Label

Value

Label

Value

Size Selector

Add to Cart Button

Success Message Banner

PROTOTYPE

BACK TO PRODUCTS



Jordan 1 Low SB UNC

★★★★★ 5.0

\$529

Lorem ipsum dolor sit, amet consectetur adipisicing elit. Numquam ullam est dicta vero sint aliquid ut accusamus, natus corporis quisquam obcaecati? Similique odio ex repellendus eaque, molestiae praesentium sunt nesciunt.

Available:

In stock

Brand:

Jordan

Model:

UNC

Sizes:

3 US

3.5 US

6 US

6.5 US

8.5 US

12 US

ADD TO CART

✔ Item added to the cart! [Visit cart](#)

f. User Profile

WIREFRAME

HEADER

Logo + Brand Name

Navigation Links

Search Bar

User + Cart

SIDEBAR NAVIGATION

PROFILE CONTENT AREA

User Greeting

Profile (Active)

Shipping Address

Purchase History

Logout

Page Title

Edit Button

First Name Field

Last Name Field

Email Field

Shoe Size Field

Username Field

Reset Password Link

PROTOTYPE

 SneakersShop

Home Shop

 CAMILLEANOVA

 Cart ¹

Hello camilleanova

Profile

Shipping Address

Purchase History

Logout

Profile

Edit

First Name

Not defined yet

Last Name

Not defined yet

Email Address

camilleo@info.com

Shoe Size(US)

8

Username

camilleanova

RESET PASSWORD

g. Cart / Complete Order Page

WIREFRAME

LOGO

BRAND NAME

NAV

NAV

search bar

SEARCH INPUT

User Info

USERNAME

Cart

Header / Navigation

Shipping Section

HEADING

Address line text

Orders Section

HEADING

Title --

Brand --

IMAGE

Size --

Quantity --

Product image

Product Name

Brand

Size: 8.5 US

Qty: 1

Price

\$000

Price Items --

Product Card

Total Row --

Order Summary Panel

HEADING

Label

\$000

Label

\$000

Label

\$00

Label

\$0

TOTAL

\$000.00

BUTTON TEXT

CTA Button

PROTOTYPE

 SneakersShop

HomeShop

Search for brand or model

CAMILLEANOVA

Cart1

SHIPPING

Address: Pacita Complex I, San Pedro, Laguna, 4024, Laguna

ORDERS



Jordan 1 Low SB UNC

UNC

Sizes: 8.5 US

Qty: 1

\$529

Order Summary

Original price

\$529.00

3 Items

\$529.00

Delivery

\$10.00

Sales tax

\$5.00

Total

\$544.00

Complete Order

h. Admin Page

WIREFRAME

PROTOTYPE

Hello Amit

+ New Item

Edit Item

Back Home

ADD A NEW ITEM

Choose an image:



Choose Fileadidas_yeezy_700_inertia.png

Shoe Title

e.g. Yeezy Boost 350 V2 Zebra

Yeezy boost inertia

Shoe Model

e.g. V2 Zebra

V2 Inertia

Shoe Price

e.g. 350

567

Shoe Brand

e.g. Yeezy

Yeezy

Shoe Colors

e.g. red,black,blue

white,orange

Shoe Size

e.g. 7,5,8,11

6.6.5

ADD NEW ITEM

REFERENCES:

<https://www.tristatetechnology.com/blog/reactjs-for-e-commerce-website-development>

<https://graphql.org/>

<https://cloudinary.com/guides/e-commerce-platform/node-js-ecommerce-benefits-architecture-and-best-practices>

<https://crystallize.com/blog/benefits-of-using-graphql>

<https://www.grandviewresearch.com/industry-analysis/sneakers-market-report>

<https://medium.com/@vrajparikh29/graphql-with-react-js-and-node-js-part-1-introduction-3105eeb3f4a4>